RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

WILDFIRE RISK PREDICITON

MLOPs Tutorial Report

submitted by

Sankalp Khamesra	**1RV23AI086**
Samruddhi D	**1RV23AI085**
Sasank Sekhar Panda	**1RV23AI087**
Shanavi Narayan	**1RV23AI089**

Artificial Intelligence and Machine Learning

2024-2025

TABLE OF CONTENTS

PHASE I

CHAPTER 1: PROBLEM STATEMENT, SCOPE AND SETUP

1.1 PROBLEM STATEMENT

Wildfires have become increasingly frequent, intense, and destructive due to climate change, prolonged droughts, rising temperatures, and changing land-use patterns. They cause significant loss of life, environmental degradation, economic damage, and long-term ecological imbalance each year. Existing wildfire management approaches are often reactive, relying heavily on manual monitoring, which limits the ability of authorities to anticipate high-risk events and take timely preventive action.

There is a critical need for an intelligent, data-driven system that can analyze historical wildfire occurrences in conjunction with environmental factors to predict wildfire risk levels.

1.2 OBJECTIVES

- To analyze historical wildfire occurrence data from the US Wildfire Dataset to identify key patterns and trends.
- To develop a predictive model that estimates wildfire risk.
- To evaluate the performance of the proposed model using appropriate accuracy and reliability metrics.

1.3 PROJECT SCOPE

The project focuses on predicting wildfire risk using historical wildfire data from the US Wildfire Dataset. The system will employ ML and DL techniques to model wildfire risk based on historical trends. The project is limited to wildfire data within the United States and does not include real-time satellite feeds or live weather data. The predictive model is intended to support decision-making and risk assessment, not as a fully autonomous alert system. The project does not involve hardware deployment or physical sensor integration and is purely software-based.

1.4 EVALUATION METRICS

- **Accuracy:** Measures the overall correctness of wildfire risk predictions by calculating the proportion of correctly classified instances.
- **Precision:** Evaluates how many predicted high-risk wildfire instances are actually true high-risk cases, helping reduce false alarms.
- **Recall (Sensitivity):** Measures the model's ability to correctly identify actual wildfire-prone or high-risk instances, which is critical for early warning and safety.
- **F1-Score:** Provides a balanced evaluation by combining precision and recall, especially useful in imbalanced wildfire datasets.
- **Confusion Matrix:** Visualizes true positives, false positives, true negatives, and false negatives to analyze model prediction behavior.
- **ROC-AUC Score:** Assesses the model's ability to distinguish between different wildfire risk levels across various classification thresholds.
- **Log Loss (Cross-Entropy Loss):** Evaluates the confidence of probability-based predictions, penalizing incorrect and overconfident predictions.

1.5 ENVIRONMENT SETUP

The project environment is set up on Windows/Linux-based systems, ensuring cross-platform compatibility. Python 3.9 or higher is used as the primary programming language for data preprocessing, machine learning model development, and backend services. Node.js (v18+) is used for frontend development and build processes. A virtual environment is employed to manage Python dependencies and ensure reproducibility, with packages installed via pip using a requirements file. Visual Studio Code is used as the primary IDE for development, and Git is utilized for version control and collaboration. The system supports CPU-based execution for standard workflows. Optional GPU acceleration is supported for deep learning models to improve training efficiency. Backend services are executed using Uvicorn for FastAPI applications. Dataset versions and trained models are tracked using DVC, ensuring reproducible experiments and controlled model lifecycle management. Docker is used to containerize both the frontend and backend inference services.

1.6 TOOLS

- **Python:** Used as the core programming language for implementing data preprocessing pipelines, feature engineering, machine learning model training, evaluation, and backend services.
- **FastAPI:** Used to deploy trained models as RESTful APIs.

- **Uvicorn:** Used as the application server to run and manage FastAPI backend services.
- **React.js (Vite):** Used to develop an interactive frontend dashboard for visualizing.
- **MLflow:** Used to track experiments, log model parameters and performance metrics, and manage multiple model versions during development.
- **DVC:** Used to version datasets and trained models, ensuring reproducibility.
- **Evidently AI:** Used to monitor data drift by comparing live prediction data with reference datasets and generating drift reports.
- **Prefect:** Used to orchestrate automated workflows, including retraining pipelines triggered by detected data drift.
- **Docker:** Used to containerize both the frontend and backend inference services.
- **Git:** Used for version control, collaboration, and maintaining the project codebase.

1.7 LITERATURE REVIEW

Wildfire risk prediction has increasingly relied on machine learning (ML) techniques due to their ability to capture complex nonlinear relationships among meteorological, topographical, vegetation, and anthropogenic variables. Traditional fire danger indices and statistical models often fail to generalize across regions and temporal scales, motivating the shift toward data-driven ML-based frameworks.

Early studies demonstrated the superiority of classical ML algorithms over conventional approaches. Cortez and Morais [1] employed Random Forest (RF) and Support Vector Machines (SVM) using meteorological variables and fuel conditions, reporting improved predictive capability compared to linear regression models. Oliveira et al. [2] extended this work by integrating land cover and topographic features, showing that RF achieved higher accuracy and area under the ROC curve (AUC) than Logistic Regression and SVM. Jaafari et al. [3] further validated the robustness of RF by comparing it with Artificial Neural Networks (ANN) and Boosted Regression Trees (BRT), where RF consistently produced AUC values exceeding 0.90 in wildfire susceptibility mapping tasks.

Ensemble learning and boosting-based methods have shown further performance gains by reducing variance and improving generalization. Pourghasemi et al. [4] demonstrated that Extreme Gradient Boosting (XGBoost) outperformed RF and SVM models, achieving AUC values above 0.92 by effectively modeling nonlinear interactions among predictors. Hong et al. [5] evaluated Gradient Boosting Machines (GBM) and RF across large forested regions and reported that GBM provided better discrimination of high-risk zones with fewer false positives. More recently, Haydar et al. [6] applied CatBoost and XGBoost to

wildfire susceptibility mapping using satellite-derived indices, achieving overall accuracies above 88% and highlighting the effectiveness of boosting models in handling categorical and heterogeneous data.

With the availability of large-scale spatiotemporal datasets, deep learning (DL) approaches have been increasingly adopted. Kondylatos et al. [7] proposed a deep neural network for next-day wildfire danger prediction using meteorological reanalysis data, reporting higher predictive skill than established fire danger indices. Jain et al. [8] employed Long Short-Term Memory (LSTM) networks to capture temporal dependencies in climate variables, achieving notable improvements over static ML models. He et al. [9] enhanced spatiotemporal modeling by integrating ConvLSTM architectures with human activity indicators, resulting in improved spatial localization of wildfire risk hotspots.

Recent advances have explored attention mechanisms and Transformer-based architectures to model long-range dependencies in wildfire processes. Li et al. [10] introduced AttentionFire, an attention-based ML framework for burned-area prediction, which achieved lower root mean square error (RMSE) compared to conventional DL models while providing improved interpretability of temporal drivers. Dubey and Dubey [11] combined Transformer networks with fuzzy inference systems, enabling stable predictions under uncertainty and translating model outputs into interpretable decision rules. Xie et al. [12] incorporated spatial attention and geographically weighted regression, demonstrating improved local-scale wildfire risk prediction in heterogeneous landscapes compared to global ML models.

The integration of remote sensing data has significantly enhanced wildfire risk modeling. Abatzoglou et al. [13] showed that satellite-derived vegetation indices and climate variables play a critical role in improving predictive performance. Durlević et al. [14] fused multi-sensor satellite data, including MODIS, Sentinel-2, and Landsat-8, using deep neural networks and achieved classification accuracies exceeding 90% for national-scale wildfire susceptibility mapping. Ye et al. [15] applied RF models trained on remote sensing and climate data to project future wildfire probability under climate change scenarios, demonstrating consistent performance across multiple projected conditions.

Several studies have emphasized ignition-specific modeling and explainability to improve operational relevance. Zhang et al. [16] focused exclusively on lightning-caused wildfires and demonstrated improved prediction precision compared to generalized wildfire models. Cui et al. [17] employed explainable machine learning techniques to analyze wildfire severity and identified anthropogenic activity as a dominant contributing factor, accounting for more than 40% of feature importance. Symeonidis et al. [18] used ensemble ML approaches to improve susceptibility mapping consistency across different ignition sources and landscape types.

Operational wildfire risk forecasting has also benefited from ML-based systems. McNorton et al. [19] introduced a global Probability-of-Fire (PoF) forecasting framework capable of producing reliable predictions up to 30 days in advance, demonstrating the scalability of ML-based fire risk models. Umunnakwe et al. [20] extended wildfire risk prediction to power system resilience planning, illustrating how ML-derived wildfire risk indices can support infrastructure protection and operational decision-making.

Overall, existing literature indicates that ensemble learning and deep learning models, particularly those incorporating spatiotemporal dependencies, remote sensing data, and explainability, represent the current state of the art in wildfire risk prediction. However, challenges remain in improving generalization across regions, handling data imbalance, and integrating uncertainty-aware decision support into operational systems.

CHAPTER 2: MLOPs, RISKS AND REQUIREMENTS ANALYSIS

2.1 UNDERSTANDING MLOPS

MLOps is a set of practices that addresses the challenges of managing the end-to-end machine learning lifecycle, including data ingestion, model development, deployment, monitoring, and continuous improvement. In our project, MLOps plays a critical role in transforming the predictive models into a production-ready decision-support system. Wildfire prediction is highly sensitive to changing environmental and climatic conditions, making continuous monitoring and retraining essential.

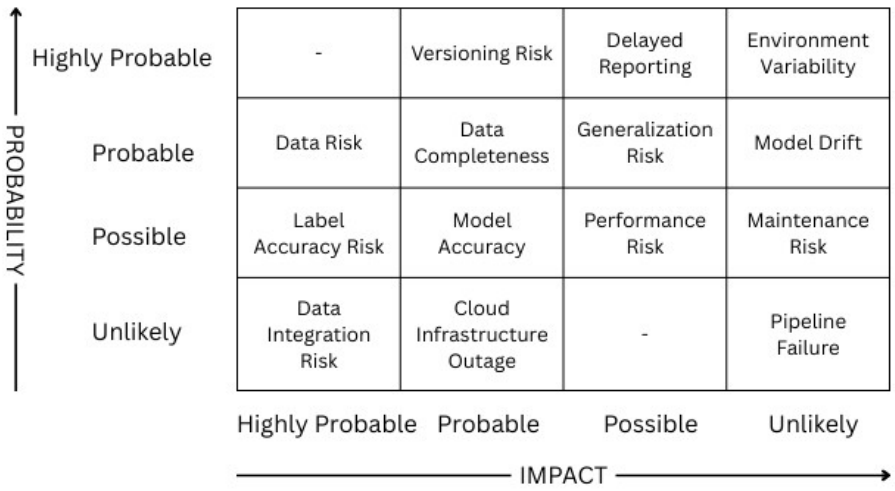
- **Data Ingestion and Versioning:** Raw wildfire data is collected from historical datasets and stored in a structured format. Dataset versions are tracked and versioned using DVC.
- **Data Preprocessing and Feature Engineering:** Data is cleaned, transformed, and engineered to extract meaningful features relevant to wildfire risk prediction.
- **Model Development and Training:** Multiple ML and DL models are trained and evaluated to identify the most effective approach for wildfire risk prediction.
- **Experiment Tracking and Model Validation:** Model parameters, evaluation metrics, and artifacts are systematically logged using MLflow to enable comparison and reproducibility.
- **Model Packaging and Deployment:** Selected models are packaged and deployed as scalable inference services, using FastAPI and Uvicorn, and containerized using Docker.
- **Monitoring and Drift Detection:** Live prediction data is continuously monitored to detect changes in data distribution or model behavior that may degrade performance. EvidentlyAI monitors live data streams to detect data drift and trigger alerts.
- **Automated Retraining and Continuous Improvement:** Retraining pipelines are triggered when performance degradation or data drift is detected, ensuring model relevance over time. Prefect orchestrates automated retraining workflows when drift thresholds are exceeded.
- **Governance and Lifecycle Management:** Models, datasets, and experiments are versioned and documented to support transparency, auditability, and responsible AI practices. Git and DVC ensure consistent tracking of code, data, and model artifacts throughout the lifecycle.

2.2 RISKS

- Data Integration Risk: Combining heterogeneous datasets like weather and meteorological records can lead to format inconsistency.
- Data Completeness Risk. Inconsistencies in the time series data like missing years and months.

- Delayed reporting of the data: The time lag in delayed reporting of the weather and meteorological data causing late assessment of the area.
- Environment Variability: Rapid and unpredictable climatic charges can reduce generalisation of the model.
- Generalization Risk: The model may not generalize to other countries and terrains.
- Label Accuracy Risk: Fires size measure can change over location affecting the class labels.
- Model Accuracy Risk: Model may confuse between controlled burns (Intentional burns) with real wildfire threats.
- Performance Risk: Model latency being slow during peak wildfire periods due to large input data.
- Model Drift: Vegetation indices such as NDVI/NBK Vary seasonally altering input patterns, due to urbanization, deforestation.
- Maintenance Risk: Model decay over time if not retrained periodically.
- Versioning Risk. Lack of version control for the data causing non-reproducible experiments.
- Data Risk: Sensor error to model from weather stations feeding data.
- Cloud Infrastructure Outage Risk
- Pipeline Failure: Failure of the MLOps pipelines during critical alert periods.

2.3 RISK MATRIX



CHAPTER 3: DATA UNDERSTANDING, FEATURE ENGINEERING AND GOVERNANCE

3.1.1 DATASET DESCRIPTION

This project utilizes the **U.S. Spatiotemporal Wildfire Ignition Dataset**, obtained from **Kaggle**, which consolidates wildfire ignition records with high-resolution meteorological data for the continental US spanning **2014 to 2025**. The dataset is derived from the wildfire ignition dataset introduced in the research paper *"Spatiotemporal Wildfire Prediction and Reinforcement Learning for Helitack Suppression"*.

The dataset integrates wildfire ignition events from **IRWIN (Integrated Reporting of Wildland Fire Information)** with daily environmental variables sourced from **GRIDMET**, a gridded surface meteorological dataset. Each data sample corresponds to a specific geographic location (latitude and longitude) and a reference date, aligned with meteorological observations and a wildfire ignition label.

To capture temporal dependencies, each sample is expanded into a 75-day time window, consisting of:

- 60 days prior to the reference date, labeled as *No Wildfire*,
- The reference day (day of ignition), labeled as *Wildfire = Yes* for positive events,
- 14 days following the ignition, also labeled as *Wildfire = Yes*.

The dataset contains a total of **126,800 labeled samples**, including **50,720 positive wildfire ignition events** and **76,080 negative samples**. Negative samples are synthetically generated using spatial and temporal offsets (near, far, and yearly offsets) to ensure robust class representation. When expanded into 75-day sequences, the dataset yields **over 9.5 million daily observation rows**, making it suitable for large-scale sequence learning and spatiotemporal analysis.

Each observation includes **15 standardized environmental variables** capturing atmospheric, fuel, and fire-risk conditions. These variables include precipitation, relative and specific humidity, solar radiation, minimum and maximum temperatures, wind speed, vapor pressure deficit, fuel moisture indices, energy release and burning indices, and evapotranspiration measures. All variables are provided in consistent and physically meaningful units to facilitate modeling and reproducibility.

3.1.2 DATA SOURCES ACKNOWLEDGMENT

- **IRWIN (Integrated Reporting of Wildland Fire Information):** Provider of wildfire ignition event labels.
- **GRIDMET:** Provider of daily gridded meteorological and environmental variables.
- **Kaggle:** Distribution platform for the processed dataset used in this project.

3.2 DATA PROFILING

A thorough data profiling was conducted to understand the dataset's structure, quality, and feature characteristics:

- **Dataset Size:** 9,509,925 daily observations with 22 attributes, including latitude, longitude, datetime, meteorological variables, fire indices, and the wildfire label.
- **Temporal Coverage:** 2014–2025
- **Spatial Coverage:** Continental United States
- **Memory footprint:** ~1.5 GB (large-scale spatiotemporal dataset)
- **Target Variable:** Binary wildfire ignition (Yes/No)
- **Class Imbalance:** Wildfire events constitute ~5.28% of total observations, with non-wildfire days accounting for ~94.72%. This indicates a strong class imbalance, which is expected in real-world wildfire ignition scenarios.
- Low missing value rates (~0.27%) across features indicate **high data quality**.
- No missing values in **target (Wildfire)** or **spatiotemporal identifiers**.

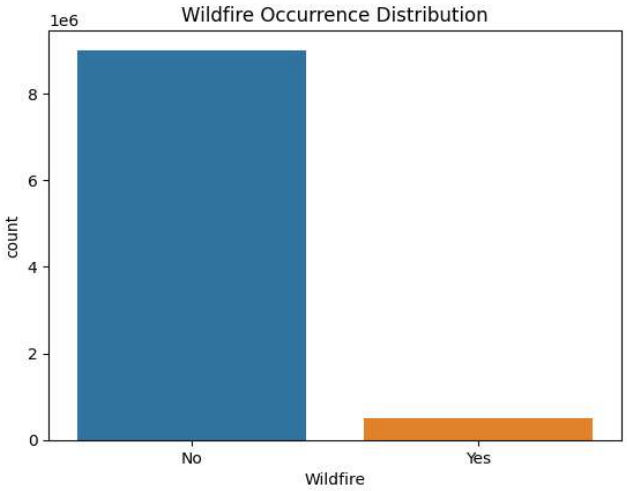


Figure 3.1: Distribution of wildfire and non-wildfire observations.

The descriptive statistics summary:

	count	mean	std	min	25%	50%	75%	max
latitude	9.510e+06	39.144	5.267	2.526e+01	34.689	38.532	43.670	48.999
longitude	9.510e+06	-106.435	14.569	-1.244e+02	-118.344	-110.926	-95.420	-67.013
pr	9.484e+06	1.737	6.330	0.000e+00	0.000	0.000	0.000	528.000
rmax	9.484e+06	76.144	20.781	5.000e+00	61.700	79.900	95.000	100.000
rmin	9.484e+06	33.524	18.362	1.000e+00	19.300	30.700	45.400	100.000
sph	9.484e+06	0.006	0.003	1.300e-04	0.004	0.006	0.008	0.024
srad	9.484e+06	221.120	88.087	0.000e+00	147.000	227.300	299.600	440.600
tmmn	9.484e+06	280.335	8.749	2.309e+02	274.800	281.100	286.300	310.000
tmmx	9.484e+06	294.301	10.239	2.419e+02	287.800	295.500	301.900	324.300
vs	9.484e+06	3.765	1.678	3.000e-01	2.600	3.400	4.600	19.600
bi	9.484e+06	33.621	23.946	0.000e+00	19.000	34.000	49.000	189.000
fm100	9.484e+06	12.787	4.949	1.100e+00	8.700	12.900	16.500	32.600
fm1000	9.484e+06	14.501	5.217	2.100e+00	10.300	14.800	18.400	42.800
erc	9.484e+06	43.573	24.154	0.000e+00	25.000	39.000	61.000	121.000
etr	9.484e+06	5.634	3.086	0.000e+00	3.200	5.400	7.700	26.500
pet	9.484e+06	4.129	2.236	0.000e+00	2.300	4.000	5.800	16.900
vpd	9.484e+06	1.132	0.882	0.000e+00	0.470	0.920	1.570	8.100

The descriptive statistics summarize approximately **9.5 million spatiotemporal observations** across geographic, meteorological, and fire-related variables. Latitude and longitude values indicate wide spatial coverage across the continental United States. Weather variables such as precipitation, temperature, humidity, wind speed, and solar radiation exhibit substantial variability, reflecting diverse climatic conditions influencing wildfire behavior. Several features, including precipitation, evapotranspiration, and fire indices, show skewed distributions with frequent zero values, characteristic of dry and low-activity periods. Fuel moisture and fire danger indices demonstrate moderate to high variance, highlighting their sensitivity to environmental changes. Overall, the statistical distribution of features confirms the dataset's richness and suitability for modeling complex wildfire risk patterns across time and space.

3.3 FEATURE EXTRACTION

In this project, we leveraged both existing meteorological and fire index features and derived temporal and principal components to capture the patterns leading to wildfire ignition.

3.3.1 Raw Meteorological and Fire Indices

The dataset includes **15 environmental variables** and fire indices from GRIDMET and IRWIN, which were used as primary predictive features:

Table 3.1: Feature Descriptions

Feature	**Description**	**Units / Type**
pr	Precipitation	mm/day
rmax, rmin	Maximum & Minimum daily relative humidity	%
sph	Specific humidity	kg/kg
srad	Solar radiation	W/m²
tmmn, tmmx	Minimum & Maximum daily temperature	°C
vs	Wind speed	m/s
bi	Burning Index	index

fm100, fm1000	Fuel moisture (100hr & 1000hr dead fuels)	%
erc	Energy Release Component	index
etr, pet	Evapotranspiration (actual & potential)	mm/day
vpd	Vapor pressure deficit	kPa

These features were chosen because **wildfire ignition is strongly influenced by temperature, humidity, wind, solar radiation, and fuel dryness**, as confirmed in scientific literature.

3.3.2 Temporal Features

To capture seasonal and interannual variations, additional features were derived from the datetime column:

- **Year:** Captures interannual variability and long-term trends in wildfire occurrence.
- **Month:** Captures seasonal patterns; wildfires are more frequent during summer.
- **Day of Year:** Captures finer temporal trends and enables modeling of seasonal cycles.

These features allow the model to learn time-dependent patterns associated with wildfire ignition.

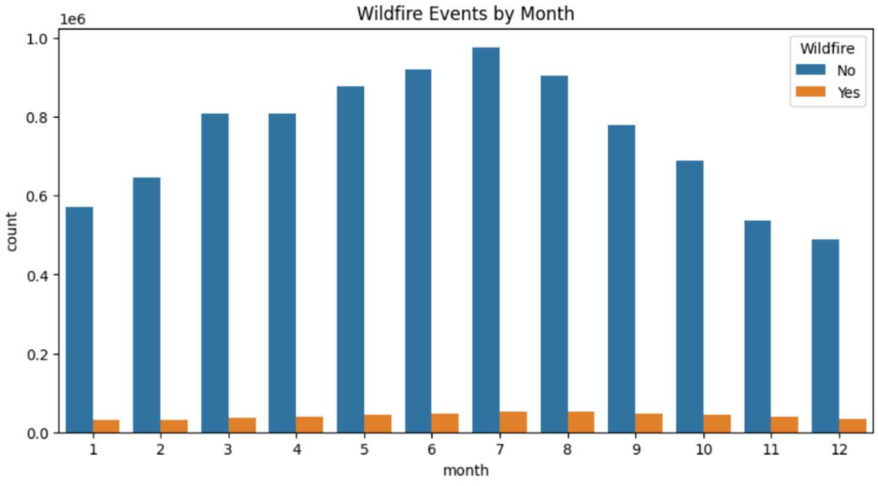


Figure 3.2: Seasonal variation of wildfire occurrences.

3.3.3 Principal Component Analysis (PCA)

Due to the **high correlation among meteorological variables** (e.g., tmmx ↔ vpd, ERC ↔ BI, fm100 ↔ fm1000), **PCA** was applied for dimensionality reduction. The variance ratio was analyzed to select components capturing most of the variance.The first few principal components captured significant variance and were added as new features to highlight latent environmental patterns associated with wildfire risk.

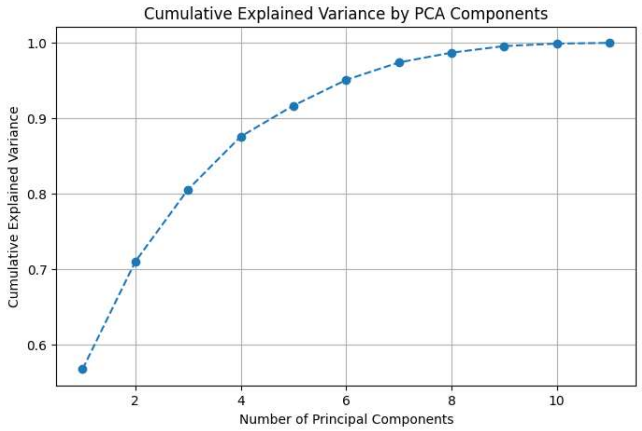


Figure 3.3: Cumulative explained variance ratio

.3.4 FEATURE SELECTION

3.4.1 Correlation-Based Filtering

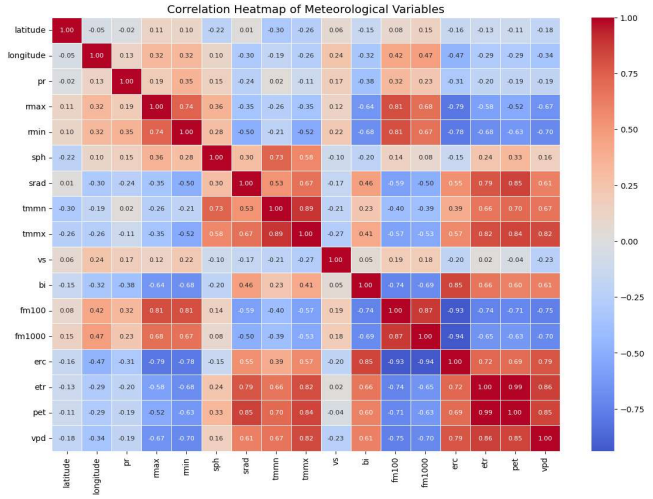


Figure 3.4 : Correlation Matrix of the features of the dataset

A correlation heatmap revealed high multicollinearity among some variables. Redundant features were either combined using PCA, or retained selectively based on domain knowledge (e.g., keeping both ERC and BI because they capture complementary fire risk information).

3.4.2 Importance from Machine Learning

A **Random Forest Classifier** was trained to evaluate feature importance for wildfire prediction. Top predictors identified:

1. **Energy Release Component (ERC)**
2. **Burning Index (BI)**
3. **Fuel Moisture (fm100, fm1000)**
4. **Temperature (tmmx, tmmn)**
5. **Vapor Pressure Deficit (vpd)**
6. **Spatial coordinates (latitude, longitude)**

These features contribute most to model decision-making and were prioritized for predictive modeling.

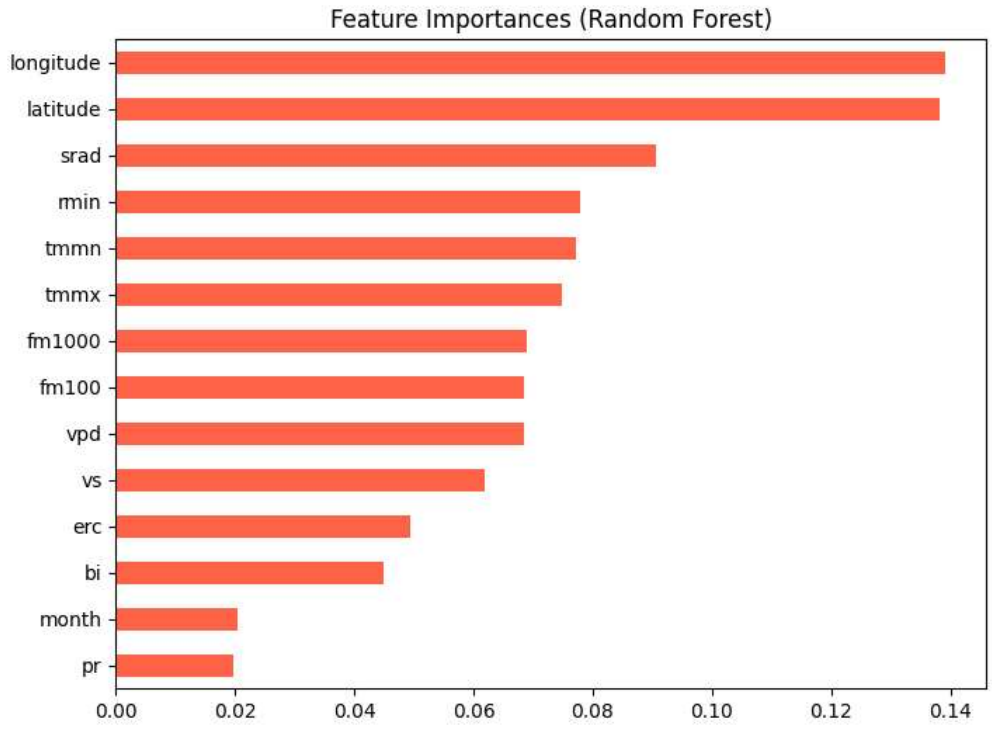


Figure 3.5: Feature importance ranking from Random Forest classifier.

3.4.3 Domain Knowledge Integration

- Features with proven physical relevance to fire ignition were retained, even if their statistical correlation with the target was moderate:
    - Wind speed (vs) → accelerates fire spread
    - Precipitation (pr) → reduces fire probability
    - Solar radiation (srad) → dries fuels

3.5 MODEL DESIGN APPROACHES

Based on insights obtained from data profiling, exploratory data analysis, and feature selection, multiple model design strategies were considered and adopted to effectively capture these complexities.Wildfire risk prediction is formulated as a binary classification problem, where each daily spatiotemporal observation is classified as:

- **1 (Wildfire)** – ignition occurred
- **0 (No Wildfire)** – no ignition occurred

Given the rarity of wildfire events (~5.28%), model design prioritizes robust detection of minority class events while minimizing false negatives. Given the heterogeneous nature of wildfire data, multiple modeling approaches are explored to balance predictive accuracy, interpretability, and operational feasibility.

Traditional machine learning algorithms are used as baseline models to establish performance benchmarks and ensure interpretability. These models are well-suited for handling non-linear relationships and feature interactions, while providing relatively faster training and inference times. To capture temporal dependencies in wildfire occurrences, deep learning architectures such as **Long Short-Term Memory (LSTM) and Convolutional LSTM (ConvLSTM)** networks are employed. These models are designed to learn sequential patterns from time-series data.

CHAPTER 4: BUILDING AND EVALUATING A MODEL

4.1 MODEL BUILDING

Initially, the cleaned and feature-engineered dataset is split into training, validation, and testing subsets. Temporal ordering is preserved where applicable to prevent data leakage, particularly for time-series modeling. Feature scaling and encoding techniques are applied as required, depending on the model architecture.

Classical machine learning models such as **Random Forest and XGBoost** are trained on structured wildfire attributes, including temporal, geographic, and environmental features. These models are configured to handle non-linear relationships and are optimized using appropriate hyperparameter tuning. For modeling temporal dependencies, **LSTM-based neural networks** are trained using sequential data representations, enabling the system to learn long-term wildfire trends. **ConvLSTM** models are further employed to capture both spatial and temporal correlations across regions.

To address class imbalance in wildfire occurrence data, resampling techniques such as **SMOTEN** are applied during training, improving the model's ability to detect high-risk wildfire events. Model training is conducted iteratively, with performance evaluated using predefined metrics and logged using MLFlow. Each training run is versioned to ensure traceability and reproducibility.

Trained models are serialized and stored as artifacts, making them suitable for deployment in production environments. The finalized models are integrated into an inference pipeline and exposed through RESTful APIs, enabling real-time or batch wildfire risk prediction.

4.2 TRAINING, VALIDATION AND TESTING

The training, validation, and testing of the wildfire risk prediction models are conducted as an **offline machine learning workflow**, separate from the deployed inference service. Model development is performed using historical wildfire data, while the production system is designed to consume only pre-trained and validated model artifacts.

During the training phase, historical wildfire data is preprocessed and used to train multiple ML and DL models. Feature engineering and resampling techniques are applied to prepare the data for learning. Training experiments are executed locally, and model parameters, metrics, and artifacts are logged using **MLflow**. Trained models are serialized and stored as versioned artifacts for later deployment.

The validation phase is used to compare model variants and tune hyperparameters prior to deployment. Validation datasets are kept separate from training data and are used to assess model stability and generalization. Performance metrics are monitored across experiments using MLflow, allowing systematic selection of the best-performing model. Only models that meet predefined performance thresholds are promoted for deployment.

Once testing is complete, the selected model is exported as a production artifact and loaded into the FastAPI-based inference service. No training or testing occurs within the deployed container; the inference service strictly operates in prediction mode. Post-deployment validation is achieved through continuous monitoring rather than retraining in production. Incoming inference data is logged and compared against reference datasets using drift detection mechanisms. When significant data drift is detected, the system triggers an offline retraining workflow, ensuring that model updates remain controlled and auditable.

4.3 EVALUATION METRICS AND VISUALIZATION

Standard classification performance metrics are computed to assess and compare different model variants. These metrics are logged and tracked using **MLflow**, enabling systematic experiment comparison and reproducibility. Metrics such as **accuracy, precision, recall. etc** are used to evaluate predictive effectiveness, with particular emphasis on recall to minimize missed high-risk wildfire events. The final selected model is promoted to deployment only after meeting predefined performance thresholds.

In the production environment, model reliability is assessed indirectly through **data drift metrics**, which evaluate whether incoming inference data deviates significantly from the reference training distribution.

MLflow provides dashboards for comparing model metrics, parameter configurations, and experiment histories. Post-deployment, **Evidently AI** is used to generate interactive HTML reports that visualize data drift, feature distribution changes, and statistical deviations between reference and live datasets. These reports provide interpretable insights into model reliability over time and serve as a trigger mechanism for retraining workflows. Additionally, a **React-based frontend dashboard** is used to present wildfire risk predictions and monitoring outputs in a user-friendly format. The frontend consumes the FastAPI inference service and displays prediction results and system status.

Figure 4.1: Test and train Accuracies of various models

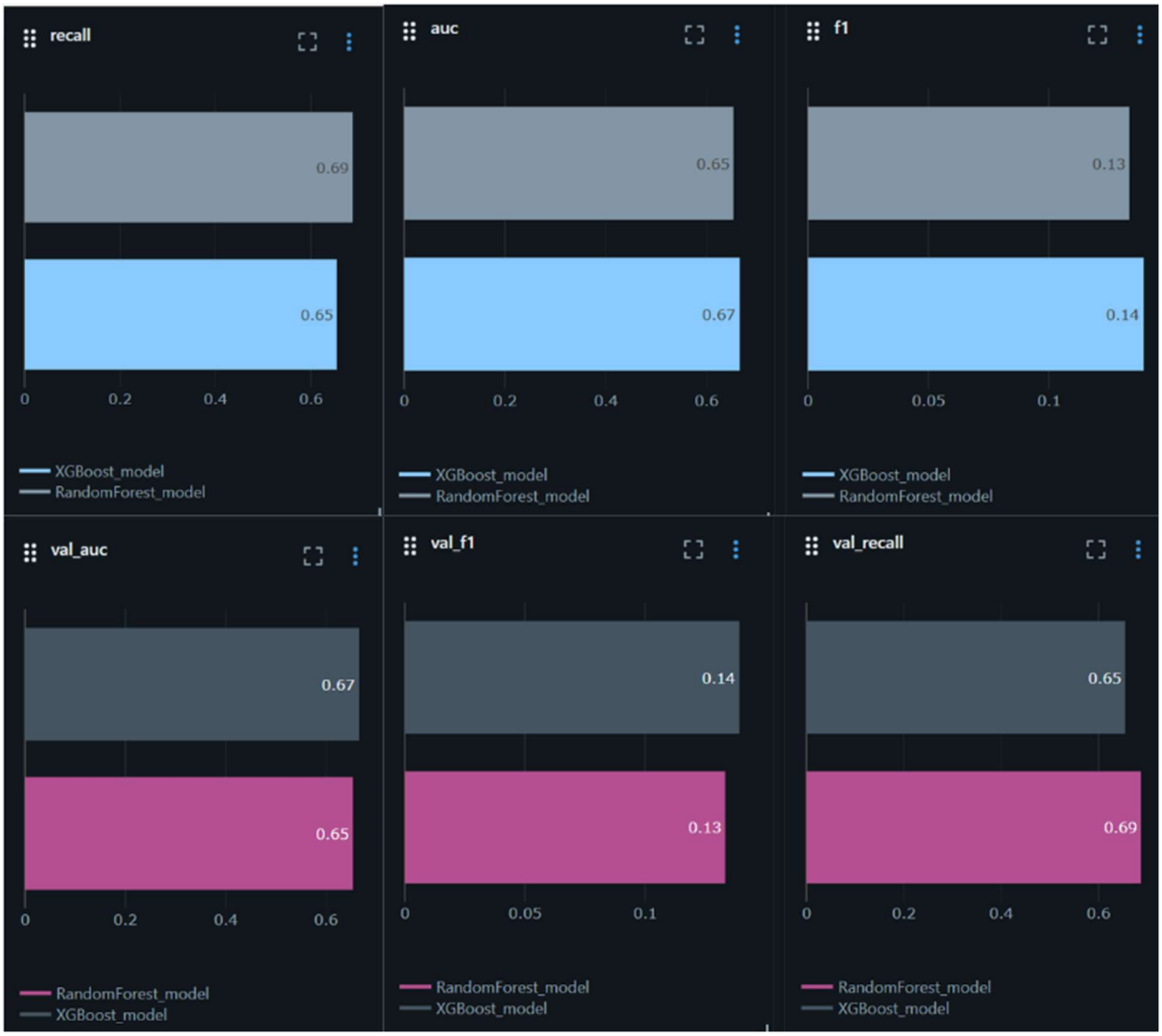


Figure 4.2 AUC, Recall, F1-score of Random Forest and XGBoost

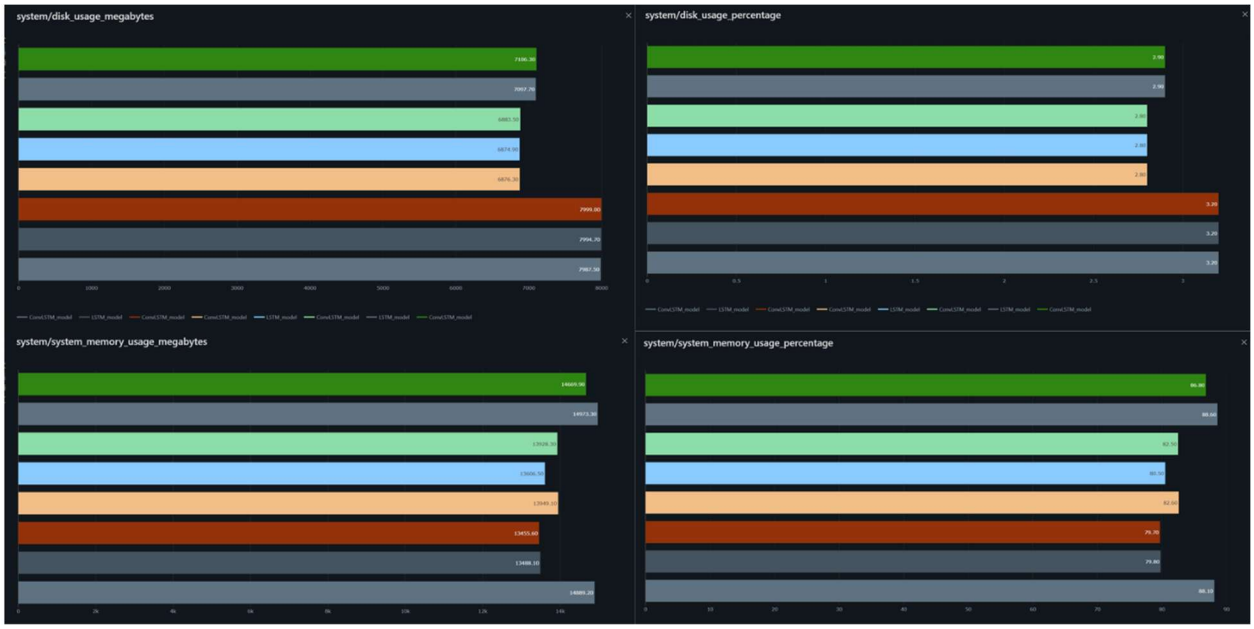


Figure 4.3 System Usage By Various Models

4.4 EXPERIMENT TRACKING

Experiment tracking is implemented to ensure reproducibility, transparency, and systematic comparison of wildfire risk prediction models. Given the separation between offline model development and online inference, all experimentation activities are conducted outside the deployed inference environment.

MLflow is used as the **primary experiment tracking tool** during the model development phase. Each training run logs key metadata including model configurations, hyperparameters, evaluation metrics, and generated artifacts. Trained models and associated preprocessing artifacts are stored as versioned outputs and referenced during deployment. Only validated and approved model artifacts are copied into the inference container, ensuring that the production system remains stable and deterministic. Experiment histories maintained through MLflow allow traceability from deployed models back to their training configurations, datasets, and performance metrics. This traceability is critical for debugging, auditing, and compliance, particularly in safety-critical applications such as wildfire risk prediction.

By decoupling experiment tracking from the inference service, the system avoids runtime overhead and unintended model changes in production, while still enabling continuous experimentation and improvement in a controlled environment.

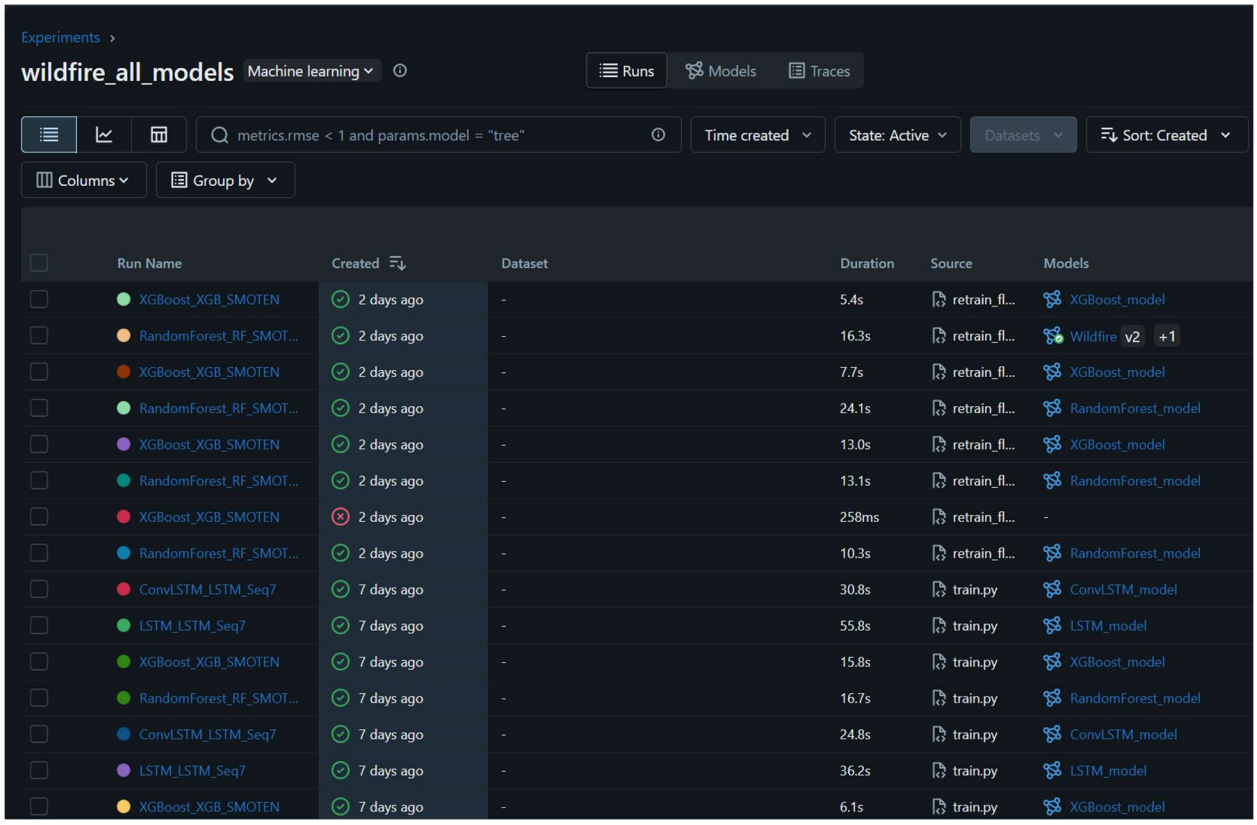


Figure 4.4 Experiment tracking of Models in MlFlow

4.5 INTERPRETING MODEL RESULTS AND ERROR ANALYSIS

Model performance is interpreted using evaluation metrics (**accuracy, precision, recall, F1-score, ROC-AUC, training/testing loss**) recorded during offline experiments and tracked through MLflow. Special attention is given to recall, as false negatives in wildfire prediction represent high-risk scenarios. Comparative analysis across multiple experiments allows identification of the most stable and generalizable model. In the deployed system, interpretation shifts from metric-based evaluation to prediction behavior analysis. Inference outputs are logged and visualized through the frontend dashboard, enabling stakeholders to observe predicted risk patterns and detect anomalies in model behavior.

Error analysis is conducted during the offline evaluation phase by examining misclassified instances in the test dataset. False positives and false negatives are analyzed to understand systematic prediction errors, such as misclassification under specific temporal or regional conditions. In the production environment, direct error computation is not performed, as ground-truth labels are not immediately available. Instead, **data drift detection** serves as a proxy for potential degradation in model accuracy. Evidently AI is used to

compare incoming inference data with the reference training distribution. When significant drift is detected, the system flags the need for offline retraining and re-evaluation.

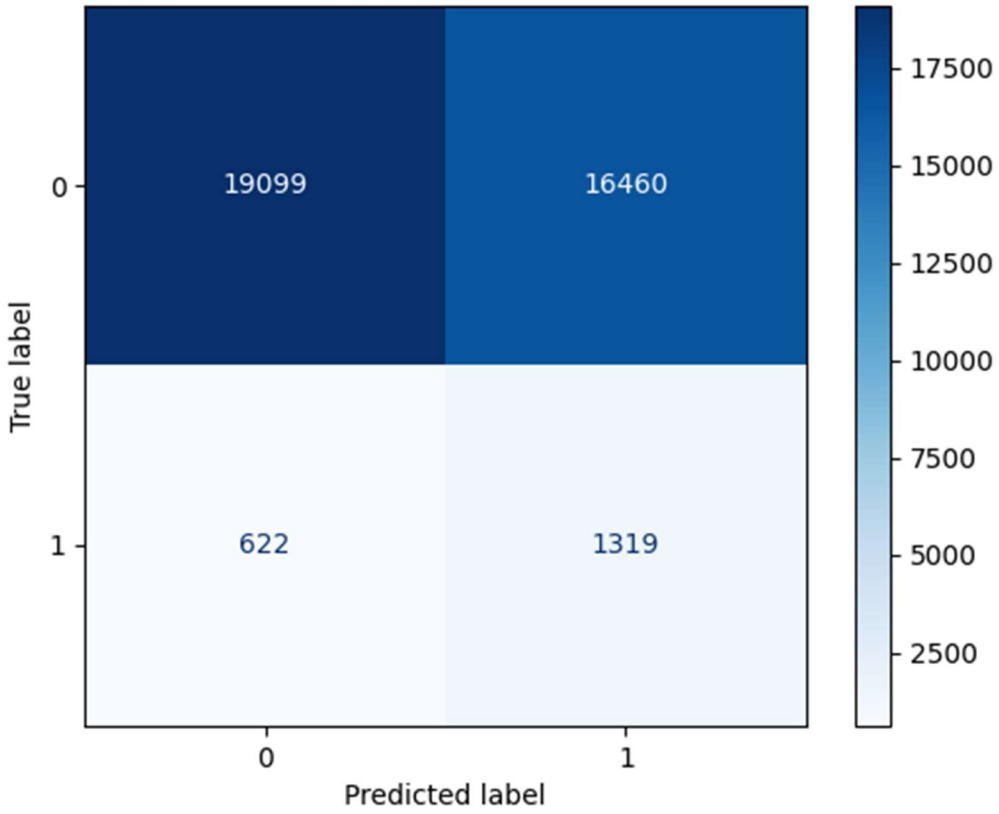


Figure 4.5 Confusion Matrix of Random Forest Model

PHASE II

CHAPTER 5: MODEL ANALYSIS AND REFINEMENT

5.1 REVISITING MODEL PERFORMANCE

Model performance in the Wildfire Risk Predictor is not treated as a one-time evaluation outcome but as a **continuously reviewed property** of the system. Given the dynamic nature of environmental and climatic data, periodic reassessment is essential to ensure sustained reliability of predictions.

During development, model performance is initially assessed and logged using MLflow. These results establish baseline performance benchmarks against which future model behavior is compared. Once deployed, the inference service operates using fixed, validated model artifacts and does not perform retraining or live metric recalculation. In the production environment, model performance is revisited indirectly through **data drift monitoring** rather than direct accuracy measurement. Incoming inference data is logged and compared with reference training data using drift detection mechanisms. When drift thresholds are exceeded, the system flags the need for **offline retraining and re-evaluation**. Updated models are then trained, validated, and tested using the same experiment tracking and evaluation pipeline. Only after demonstrating improved or stable performance relative to previous benchmarks are new models promoted to deployment.

5.2 HYPERPARAMETER TUNING AND OPTIMIZATION

Hyperparameter tuning and optimization are performed as part of the **offline model development workflow**, prior to deployment. Since the deployed system operates strictly in inference mode, all optimization activities are completed outside the production environment to ensure stability and reproducibility. This design choice prevents unintended model behavior changes during inference.

During model development, different hyperparameter configurations are explored across multiple training runs. These configurations include model-specific parameters. Each experiment is executed independently and logged using **MLflow**, capturing hyperparameter values alongside corresponding evaluation metrics. Performance comparisons across experiments enable identification of configurations that provide the best trade-off between predictive effectiveness and model stability.

Once an optimal configuration is identified, the resulting model is evaluated on a held-out test dataset to confirm generalization performance. Only models that demonstrate consistent improvement or stable

performance are selected as final artifacts. These optimized models are then serialized and deployed within the inference service.

5.3 INTERPRETABILITY

During the offline development phase, interpretability is achieved primarily through **model performance analysis and comparative experimentation**. By training and logging the model performance metrics, insights are gained into how different model configurations respond to changes in data and features.

In the deployed system, interpretability is emphasized at the **system-output level rather than the internal model level**. The inference service exposes clear wildfire risk predictions through a REST API, which are visualized in the frontend dashboard. This allows users to interpret results based on predicted risk patterns and trends, rather than raw model internals. Interpretability is achieved through transparent evaluation, monitored behavior, and clear visualization of outputs and data trends.

5.4 MODEL VERSIONING AND REGISTRY

During the offline experimentation phase, **MLflow** is used to log trained models along with their associated parameters, metrics, and artifacts. Each training run produces a uniquely identifiable model version, enabling comparison across experiments and maintaining a complete history of model evolution. Rather than dynamically updating models in production, the project follows an **artifact-based promotion strategy**. Once a model demonstrates satisfactory performance on validation and test datasets, its serialized artifacts are exported and stored in a designated artifacts directory. These artifacts are then explicitly included in the inference container during the Docker build process.

Any model update requires retraining, re-evaluation, and rebuilding of the inference container. Version control for supporting assets such as datasets and preprocessing outputs is maintained using **DVC**, while source code versioning is handled through **Git**. Together, these tools provide end-to-end traceability across code, data, and model versions.

5.5 DOCUMENTATION OF MODEL UPDATES

All updates originate from the offline training and experimentation phase. Each training run is logged using MLflow, which automatically captures model parameters, evaluation metrics, run metadata, and generated artifacts. This creates a permanent and timestamped record of every candidate model version.

When a model demonstrates improved or acceptable performance, it is explicitly exported as a versioned artifact along with its corresponding preprocessing artifacts. These files represent a documented snapshot of the model state, including feature ordering, scaling logic, and learned parameters.

Any update to the deployed model requires a new inference build, where the updated artifacts are copied into the inference service container. The inference API exposes the active model name through a dedicated endpoint, allowing operational verification of which model version is currently serving predictions. Model updates triggered by data drift detection are also documented. Drift reports generated using **EvidentlyAI** are stored as HTML and JSON artifacts, along with a machine-readable status file indicating whether retraining was recommended. If retraining occurs, the retraining pipeline execution, resulting metrics, and newly registered model artifacts serve as documented evidence of the update rationale.

Supporting assets such as datasets and preprocessing outputs are versioned using **DVC**, while all source code changes related to training, inference, or monitoring are tracked via **Git**. Together, these mechanisms provide a complete historical record explaining what changed, why it changed, and how the new model was produced.

CHAPTER 6. IMPLEMENTING CI/CD PIPELINE

6.1 OVERVIEW OF CI/CD FOR ML

CI/CD for ML extends traditional software CI/CD by incorporating **model training, validation, evaluation, and deployment workflows** in addition to code integration. Unlike standard applications, ML systems must handle evolving data, model artifacts, performance metrics, and reproducibility requirements. In this project, CI/CD ensures that any change to model code, inference logic, or configuration is automatically validated, tested, and safely deployed.

6.2 TOOLS SETUP: GITHUB ACTIONS

GitHub Actions is used as the CI/CD orchestration tool for this project due to its tight integration with the source code repository and ease of automation.

- Executes automated checks for training, inference, and deployment logic
- Builds and validates Docker images for inference and frontend services
- Enforces consistency between code, model artifacts, and deployment environment

6.3 AUTOMATING MODEL TRAINING AND TESTING

Model training automation is implemented as a controlled CI job, rather than continuous retraining on every commit.

1. **Dependency Installation**
    - Python environment initialized using *requirements.txt and requirements.inference.txt*.
2. **Model Training Execution**
    - The training script is executed with fixed seeds for reproducibility.
    - Hyperparameters and metrics are logged using MLflow.
3. **Model Evaluation**
    - Evaluation metrics are validated against predefined acceptance thresholds.
    - Training fails automatically if metrics fall below required performance.
4. **Artifact Validation**
    - Generated model and preprocessing artifacts are checked for correctness.
    - Artifacts are stored in the *artifacts/* directory for downstream use.

This automation ensures that only valid and performance-verified models move forward in the pipeline.

6.4 BUILDING AND RUNNING A DEPLOYMENT PIPELINE

The deployment pipeline is fully **container-driven** and aligned with our Docker Compose setup.

- **Docker Image Build:** Inference service image built using *python:3.10-slim*. Frontend image built using *nginx:stable-alpine*.
- **Artifact Injection:** Versioned model artifacts are copied into the inference container.
- **Container Validation:** Inference API health check is executed. Model loading and prediction endpoints are tested.
- **Service Orchestration:** Docker Compose spins up *Inference API (port 8000)* and *Frontend service (port 8001)*.
- **Rollback Safety:** Previous images remain available for fast rollback if deployment fails.

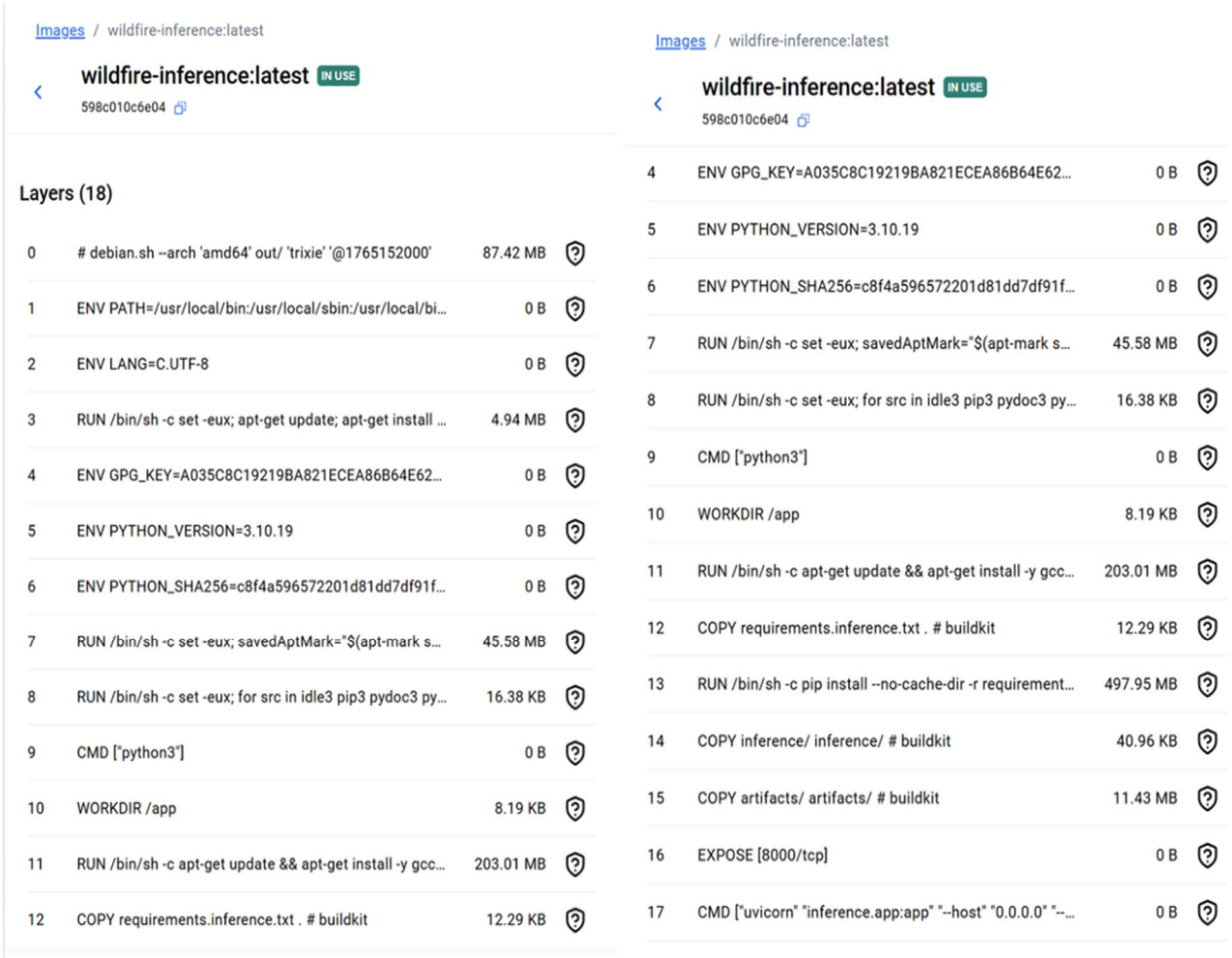


Figure 6.1 Layers of the Docker Image build

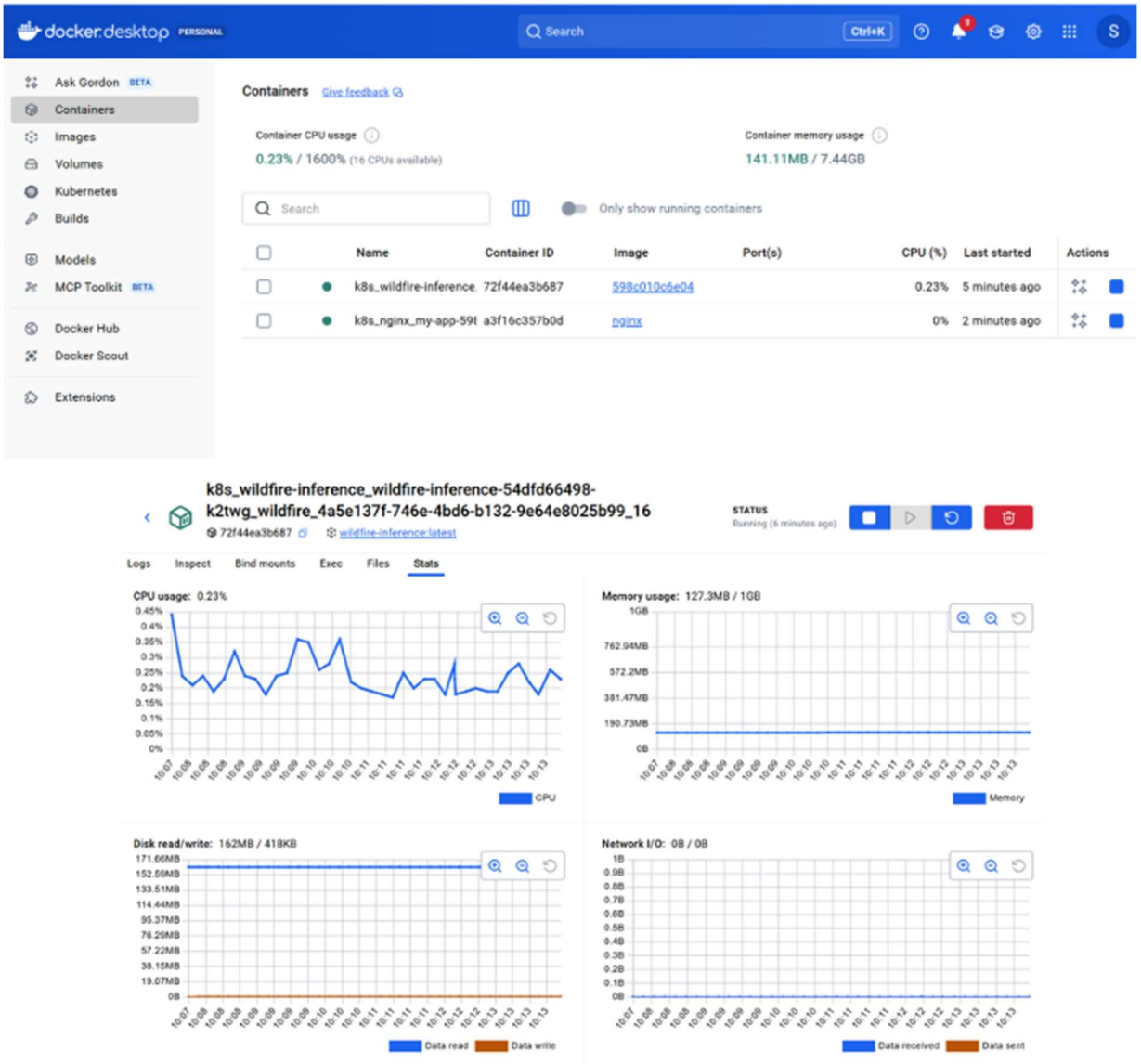


Figure 6.2 Performance Stats of the Docker Image

CHAPTER 7: MONITORING AND TRACKING MODEL LIFECYCLE

7.1 MODEL MONITORING CONCEPTS AND KPIs

Model monitoring ensures that the wildfire risk prediction system continues to perform reliably after deployment. Since real-world environmental patterns and climate variables change over time, monitoring focuses not only on system health but also on model behavior and prediction quality. The KPIs are:

- **System-Level KPIs:** API uptime and availability, Request latency and throughput, Error rates (HTTP 4xx / 5xx), Resource usage (CPU, memory).
- **Model-Level KPIs:** Prediction confidence distribution, Class distribution of wildfire risk outputs, Input feature statistics (mean, variance, missing values), Model response stability over time.

7.2 MONITORING TOOLS USED

Prefect is responsible for scheduling monitoring jobs, executing drift detection pipelines, managing task dependencies and retries and logging workflow execution states. Prefect provides control and automation across the ML monitoring lifecycle.

EvidentlyAI is used for data drift and model behavior analysis. It operates on logged inference data stored via mounted volumes.

Capabilities include:

- Input data drift detection
- Prediction distribution shifts
- Feature-level statistical comparison

7.3 DETECTING DATA DRIFT AND CONCEPT DRIFT

Data drift is identified by comparing statistical properties of incoming inference data against a reference dataset used during training. Monitored aspects include: Feature distribution shifts, Changes in categorical value frequencies and Increase in missing or anomalous values. EvidentlyAI reports highlight these deviations, signaling when input data no longer resembles training data.

Concept drift occurs when the relationship between input features and predicted wildfire risk changes over time. Detection is performed indirectly by: Monitoring prediction confidence decay, Tracking output class imbalance over time and Observing sudden behavioral changes in similar input patterns.

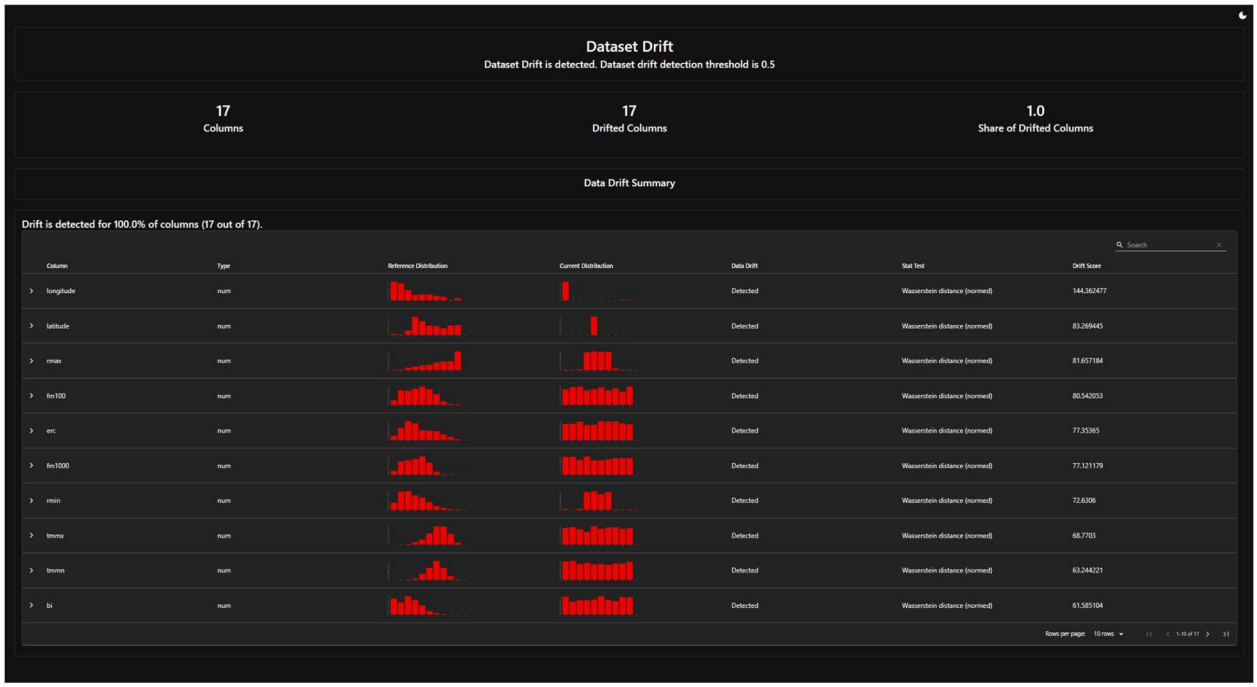


Figure 7.1 Evidently AI Drift detection

7.4 MODEL RETRAINING AND REDEPLOYMENT STRATEGY

The project follows a manual-triggered retraining strategy to ensure reliability and governance.

- Drift alerts or performance degradation is detected
- Training pipeline is executed offline
- New model is evaluated against baseline metrics
- Model artifacts are logged and versioned using MLflow
- Approved artifacts are exported for deployment
- Inference Docker image is rebuilt with updated artifacts
- Services are redeployed using Docker Compose

This controlled process prevents unverified models from entering production.

7.5 LOGGING, ALERTING AND DASHBOARD SETUP

Inference requests, predictions, and feature summaries are logged persistently using mounted volumes. These logs serve as Input for EvidentlyAI reports, Audit trails for predictions and Debugging resources for failures.

Alerts are configured based on Latency thresholds, Error rate spikes and Significant drift detection events. These alerts notify maintainers when immediate investigation or retraining is required. The dashboards provide Real-time inference monitoring and drift visualization.

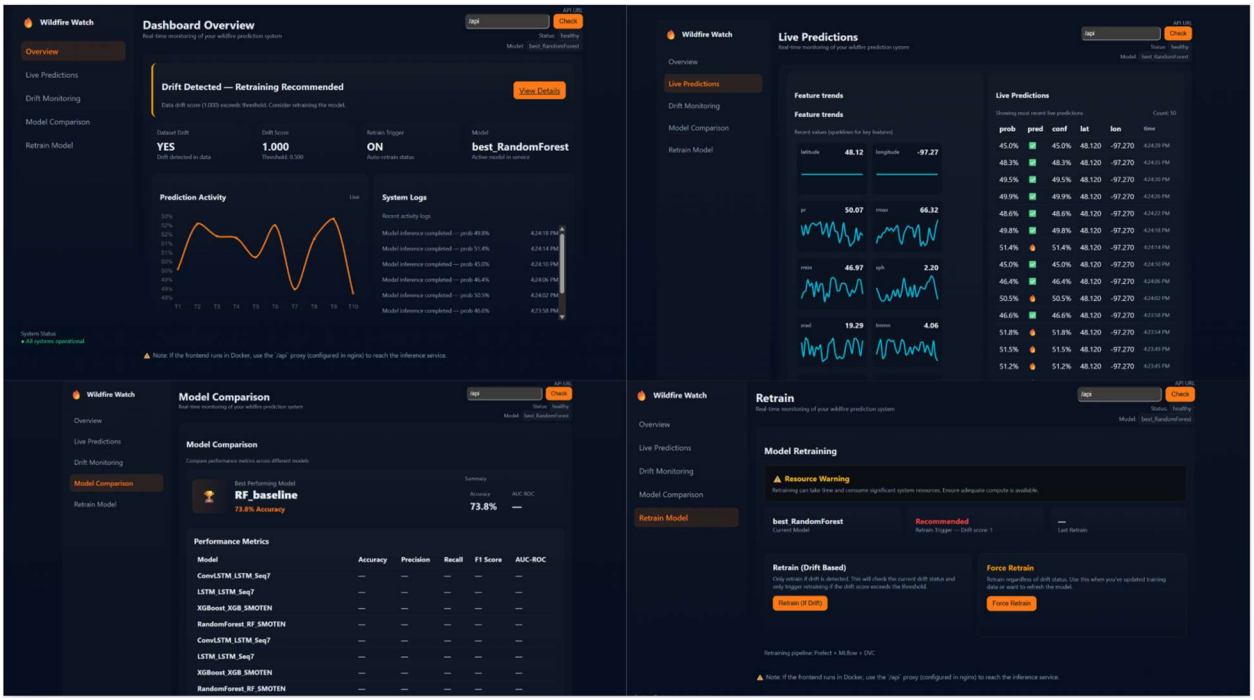


Figure 7.2 Dashboard UI for the Wildfire Risk Prediction

CHAPTER 8: PERFORMANCE EVALUATION AND REVIEW

8.1 POST DEPLOYMENT PERFORMANCE REVIEW

Post-deployment performance evaluation is conducted to ensure that the deployed wildfire risk prediction model continues to meet reliability, accuracy, and operational expectations under real-world conditions. Since live labels are not immediately available, performance assessment relies on **proxy metrics**, **prediction behavior analysis**, and **drift indicators** rather than direct accuracy computation.

Using Prefect-orchestrated monitoring workflows, logged inference data is periodically reviewed to analyze:

- Stability of prediction distributions
- Confidence score trends over time
- Input feature behavior relative to training baselines
- Operational KPIs such as latency and error rates

These reviews help identify silent performance degradation that may not trigger system-level failures but could affect decision quality in risk-sensitive scenarios such as wildfire preparedness.

8.2 CONTINUOUS IMPROVEMENT VIA FEEDBACK LOOPS

The project implements a closed-loop improvement mechanism that connects monitoring, evaluation, and retraining.

- Inference data and prediction logs are collected continuously
- Prefect workflows periodically execute drift and behavior analysis
- EvidentlyAI reports highlight deviations from training distributions
- Performance signals are reviewed against predefined thresholds
- Retraining is triggered manually or programmatically when required
- Improved models are evaluated, versioned, and redeployed

8.3 GOVERNANCE AND ETHICAL AI PRACTICES

Given the societal and environmental impact of wildfire risk predictions, governance and ethical considerations are embedded into the system design.

- **Human-in-the-loop control**: All retraining and deployment decisions require explicit approval.
- **Reproducibility**: Every deployed model can be traced back to its training data, parameters, and evaluation results
- **Stability over automation**: Automatic retraining is avoided to prevent unverified model behavior.
- **Explainability awareness**: Prediction outputs are accompanied by confidence indicators and interpretability artifacts

8.4 DOCUMENTATION

Structured documentation is maintained to support transparency, auditability, and stakeholder communication. Each deployed model is accompanied by a model card documenting:

- Model objective and scope
- Input features and assumptions
- Training data summary
- Evaluation metrics and limitations
- Deployment version and date
- Known failure modes and risks

Model cards are generated alongside model artifacts and stored with versioned releases.

8.5 AUDIT AND REVIEW CHECKLIST

8.5.1 Pre-Deployment Review

A comprehensive pre-deployment review is conducted to ensure technical correctness, reliability, and governance compliance. Model performance is verified against predefined acceptance thresholds using validated evaluation metrics. Drift analysis is performed by comparing training data distributions with recent reference datasets to confirm data consistency. All model artifacts, including trained weights and preprocessing components, are properly versioned and logged to ensure traceability and reproducibility. In addition, complete model cards and data cards are prepared to document model behavior, assumptions, and limitations. Finally, the inference container is rigorously tested to confirm successful model loading, API responsiveness, and compatibility with the deployment environment.

8.5.2 Post-Deployment Review

A structured post-deployment review is carried out to monitor system stability and model behavior in real-world conditions. Monitoring dashboards are validated to ensure continuous visibility into system health and inference performance. Prediction outputs are analyzed to confirm behavioral stability and absence of abnormal distribution shifts. All alerts and failure signals are reviewed, ensuring that no unresolved issues remain in the production system. Drift detection reports generated during operation are reviewed and archived for audit purposes. Additionally, rollback mechanisms are verified to ensure that the system can safely revert to a previous stable model version if required.

APPENDIX

A.1 LIST OF MLOPS TOOLS, ITS FEATURES, PROS AND CONS

1. MLflow: Experiment tracking, model versioning, and artifact management

Key Features

- Logs parameters, metrics, and model artifacts
- Enables reproducibility of model training
- Supports model comparison and evaluation

Pros

- Lightweight and easy to integrate
- Strong support for experiment reproducibility
- Clear traceability between models and metrics

Cons

- Model registry features require additional setup
- Not designed for real-time monitoring

2. Prefect: Workflow orchestration for monitoring, drift detection, and retraining

Key Features

- Task and flow-based pipeline orchestration
- Scheduling and retry mechanisms
- Observability of ML workflows
- Trigger-based pipeline execution

Pros

- Highly flexible and Python-native
- Easy to debug and monitor workflows
- Strong support for conditional execution
- Ideal for complex ML pipelines

Cons

- Needs external tools for visualization and metrics
- Adds orchestration overhead for small pipelines

3. Docker: Containerization of inference service and frontend

Key Features

- Environment consistency across development and production
- Isolated runtime for inference and frontend
- Image-based deployment
- Dependency encapsulation

Pros

- Ensures reproducible deployments
- Simplifies environment setup
- Reduces system compatibility issues
- Industry-standard deployment practice

Cons

- Requires container management expertise
- Debugging containers can be complex
- Slight performance overhead

4. EvidentlyAI: Data drift and model behavior analysis

Key Features

- Data drift detection
- Prediction distribution analysis
- Feature-level statistical comparison
- Automated report generation

Pros

- ML-specific monitoring capabilities

- Easy integration with logged data
- Clear, interpretable reports
- Open-source friendly

Cons

- Not real-time by default
- Computationally expensive for large datasets

5. Git & GitHub: Source code version control and collaboration

Key Features

- Code versioning
- Branching and merging
- CI/CD pipeline triggers
- Collaboration support

Pros

- Industry-standard version control
- Enables CI/CD automation
- Strong collaboration features

Cons

- No native model or data versioning
- Requires discipline in workflow management
- Limited visibility into ML artifacts

6. DVC (Data Version Control): Dataset and artifact versioning

Key Features

- Data versioning alongside code
- Lightweight tracking of large datasets
- Pipeline reproducibility
- Storage abstraction

Pros

- Enables reproducible ML experiments
- Integrates well with Git
- Scalable for large datasets

Cons

- Initial setup complexity
- Requires external storage backend

A.2 TOOL INSTALLATION GUIDES

1. **MLFlow**
   - Create and activate a virtual environment
   - Install MLflow using `pip`
   - Verify installation by running the MLflow CLI and checking the version
2. **DVC**
   - Install DVC using `pip install dvc`
   - Initialize DVC inside the Git repository using `dvc init`
   - Configure a local or remote storage backend (optional)
3. **PREFECT**
   - Install Prefect using `pip install prefect`
   - Verify installation by running `prefect version`
   - Initialize Prefect locally (optional) for UI and flow tracking.
4. **EVIDENTLY AI**
   - Install EvidentlyAI using `pip install evidently`
   - Import Evidently modules within drift detection scripts
5. **DOCKER**
   - Download Docker Desktop (Windows/macOS) or Docker Engine (Linux)
   - Enable Docker daemon
   - Verify installation using `docker --version`
6. **GIT & GITHUB**
   - Install Git using OS-specific installers
   - Configure Git with username and email

- Create a GitHub repository and link it to the local project

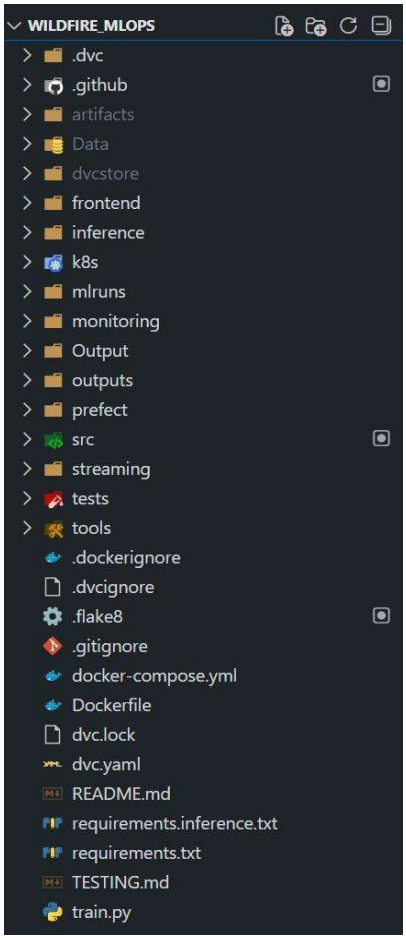
A.3 DATASET REFERENCES AND LICENSING

The dataset used in this project was obtained from Kaggle: US Wildfire Dataset (2014-2025) Available at: https://www.kaggle.com/datasets/firecastrl/us-wildfire-dataset. This dataset is derived from the research paper: *Spatiotemporal Wildfire Prediction and Reinforcement Learning for Helitack Suppression, Proceedings of the International Conference on Machine Learning and Applications (ICMLA), 2025.*

The wildfire ignition labels originate from IRWIN (Integrated Reporting of Wildland Fire Information), and the meteorological variables are sourced from GRIDMET (Gridded Surface Meteorological Dataset).

The dataset is publicly available on Kaggle and is provided for research and educational use. Usage of the dataset is permitted under Kaggle's dataset sharing terms, subject to proper attribution to the original authors and sources. All data usage in this project complies with the dataset's licensing terms and ethical research practices.

A.4 FOLDER STRUCTURE



A.5 REFERENCES

[1] P. Cortez and A. Morais, "A data mining approach to predict forest fires," *Expert Systems with Applications*, vol. 36, no. 6, pp. 984–995, 2009, doi: 10.1016/j.eswa.2008.10.003.

[2] S. Oliveira, F. Oehler, J. San-Miguel-Ayanz, A. Camia, and J. M. C. Pereira, "Modeling spatial patterns of fire occurrence in Mediterranean Europe using multiple regression and machine learning," *Science of the Total Environment*, vol. 580, pp. 136–146, 2017, doi: 10.1016/j.scitotenv.2016.11.193.

[3] A. Jaafari, M. Mafi-Gholami, P. Gholami, and H. R. Pourghasemi, "Comparison of machine learning techniques for wildfire susceptibility mapping," *Remote Sensing*, vol. 11, no. 4, Art. no. 453, 2019, doi: 10.3390/rs11040453.

[4] H. R. Pourghasemi, A. Gayen, T. Panahi, and C. Rezaie, "Wildfire susceptibility mapping using XGBoost and Random Forest," *Ecological Indicators*, vol. 118, Art. no. 106731, 2020, doi: 10.1016/j.ecolind.2020.106731.

[5] H. Hong, S. Pourghasemi, M. Shirzadi, and T. Tiefenbacher, "Forest fire susceptibility mapping using ensemble learning methods," *Environmental Modelling & Software*, vol. 134, Art. no. 104832, 2020, doi: 10.1016/j.envsoft.2020.104832.

[6] M. Haydar, M. Rahman, and S. Hossain, "Data-driven forest fire susceptibility mapping using machine learning techniques," *Ecological Indicators*, vol. 166, Art. no. 112264, 2024, doi: 10.1016/j.ecolind.2024.112264.

[7] S. Kondylatos, J. J. McCarty, and E. Foufoula-Georgiou, "Wildfire danger prediction and understanding with deep learning," *Geophysical Research Letters*, vol. 49, no. 17, 2022, doi: 10.1029/2022GL099368.

[8] P. Jain, S. Coogan, S. Subramanian, and M. Flannigan, "Spatiotemporal wildfire prediction using LSTM networks," *Natural Hazards*, vol. 106, pp. 1–19, 2021, doi: 10.1007/s11069-020-04405-7.

[9] Z. He, J. Zhang, and Y. Wang, "Deep learning modeling of human-activity-affected wildfire risk by incorporating structural features," *Ecological Indicators*, vol. 160, Art. no. 111946, 2024, doi: 10.1016/j.ecolind.2024.111946.

[10] F. Li, S. Venevsky, and Y. Ciais, "AttentionFire: An interpretable machine learning model for burned-area prediction," *Geoscientific Model Development*, vol. 16, pp. 869–889, 2023, doi: 10.5194/gmd-16-869-2023.

[11] P. Dubey and P. Dubey, “Bridging spatiotemporal wildfire prediction and decision modeling using transformer networks and fuzzy inference systems,” *MethodsX*, vol. 15, Art. no. 103498, 2025, doi: 10.1016/j.mex.2025.103498.

[12] S. Xie, H. Xiao, G. Zhang, and H. Xu, “A spatiotemporal wildfire risk prediction framework integrating density-based clustering and GTWR–RFR,” *Forests*, vol. 16, no. 11, Art. no. 1632, 2025, doi: 10.3390/f16111632.

[13] J. T. Abatzoglou, C. A. Williams, and R. Barbero, “Global emergence of anthropogenic climate change in fire weather indices,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 42, pp. 11770–11775, 2016, doi: 10.1073/pnas.1607171113.

[14] U. Durlević, V. Ilić, and A. Valjarević, “Wildfire susceptibility mapping using deep learning and machine learning models based on multi-sensor satellite data fusion,” *Fire*, vol. 8, no. 10, Art. no. 407, 2025, doi: 10.3390/fire8100407.

[15] Y. Ye, X. Chen, and L. Zhang, “Projecting spatiotemporal forest fire probability under climate change scenarios,” *Fire Ecology*, vol. 21, no. 2, 2025, doi: 10.1186/s42408-025-00433-9.

[16] Q. Zhang, C. Gao, and C. Shi, “An improved machine-learning model for lightning-ignited wildfire prediction,” *Environmental Research Letters*, vol. 20, no. 6, 2025, doi: 10.1088/1748-9326/add754.

[17] S. Cui, Y. Liu, and Z. Wang, “Explainable machine learning identifies anthropogenic activity as a key driver of forest fire severity,” *GIScience & Remote Sensing*, vol. 62, no. 1, 2025, doi: 10.1080/15481603.2025.2599489.

[18] P. Symeonidis, T. Vafeiadis, D. Ioannidis, and D. Tzovaras, “Wildfire susceptibility mapping using ensemble machine learning,” *Earth*, vol. 6, no. 3, Art. no. 75, 2025, doi: 10.3390/earth6030075.

[19] J. McNorton, S. Di Giuseppe, and E. Pappenberger, “Global probability-of-fire forecasting using machine learning,” *Geophysical Research Letters*, 2024, doi: 10.1029/2023GL107929.

[20] A. Umunnakwe, M. Khorrami, and W. Li, “Wildfire risk prediction for power system resilience using machine learning,” *IET Generation, Transmission & Distribution*, vol. 16, no. 8, pp. 1534–1545, 2022, doi: 10.1049/gtd2.12463.

A.6 FURTHER READING

1. DVC – https://dvc.org
2. MLflow – https://mlflow.org
3. Prefect – https://www.prefect.io
4. GitHub Actions – https://docs.github.com/actions
5. Docker – https://docs.docker.com
6. Kubernetes – https://kubernetes.io
7. Evidently AI – https://www.evidentlyai.com

DATASET LINK: https://www.kaggle.com/datasets/firecastrl/us-wildfire-dataset

EDA LINK: https://www.kaggle.com/code/sankalp234/wildfire-risk

GITHUB LINK: https://github.com/itsthemahn/Wildfire_risk